Január 31-én Budapesten ünnepeljük a Wikipédia 25. születésnapját. Gyere el te is! [elrejt]

Programtervezési minta

A Wikipédiából, a szabad enciklopédiából

Tartalomjegyzék elrejtés
Bevezető
A programtervezési minták fogalma
A programtervezési minták definíciója, helye
Szerkezete
Tervezési minták osztályozása
Dokumentáció
Elemzés
Jegyzetek
Források
Fordítás

Az informatikában a **programtervezési mintának** (angolul *Software Design Patterns*) nevezik a gyakran előforduló programozási feladatokra adható általános, újrafelhasználható megoldásokat. Egy programtervezési minta rendszerint egymással együttműködő objektumok és osztályok leírása.[1]

A tervminták nem nyújtanak kész tervet, amit közvetlenül be lehet kódolni, habár vannak hozzájuk példakódok, amiket azonban meg kell tölteni az adott helyzetre alkalmas kóddal. Céljuk az, hogy leírást vagy sablont nyújtsanak. Segítik formalizálni a megoldást.

A minták rendszerint osztályok és objektumok közötti kapcsolatokat mutatnak, de nem specifikálják konkrétan a végleges osztályokat vagy objektumokat. A modellek absztrakt osztályai helyett egyes esetekben interfészek is használhatók, habár azokat maga a tervminta nem mutatja. Egyes nyelvek beépítetten tartalmaznak tervmintákat. A tervminták tekinthetők a strukturált programozás egyik szintjének a paradigma és az algoritmus között.

A legtöbb tervminta objektumorientált környezetre van kidolgozva. Mivel a funkcionális programozás kevéssé ismert és használt, arra a környezetre még csak kevés tervminta ismert, például a monád. Az objektumorientált minták közül nem mindegyiket lehet, és nem mindegyiket érdemes itt használni. Van, amit módosítani kell.

A programtervezési minták fogalma [szerkesztés]

A programtervezési minták fogalma Christopher Alexander építész ötlete nyomán született meg. Ő volt az, aki olyan, az építészetben újra és újra felbukkanó mintákat keresett, amelyek a jól megépített házakat jellemzik. Könyvében, a „The Timeless Way of Buliding”-ben olyan mintákat próbált leírni, amelyek segítségével akár egy kezdő építész is gyorsan jó épületeket tervezhet. A minták a létrehozó építészek sok éves tapasztalata miatt szebb, jobb vagy használhatóbb házakat eredményeztek, mintha a tervezőnek csupán saját erejére támaszkodva kellett volna megterveznie azokat.

1987-ben Kent Beck és Ward Cunningham elkezdett kísérletezni hasonló minták alkalmazásával a programozásban, speciális mintanyelvek segítségével. Eredményeiket még abban az évben az OOPSLA konferencián mutatták be.[2] A következő években kutatásaikhoz mások is társultak.[3]

A programtervezési minták a 90-es évek elején (1994) tettek szert népszerűségre, amikor a négyek bandájaként vagy gammáékként (angolul **"Gang of Four"** vagy **GoF**)-ként emlegetett Erich Gamma, Richard Helm, Ralph Johnson és John Vlissides programozó négyes kiadta a Programtervezési minták[4] című könyvüket, amely ma is alapjául szolgál az objektumorientált programozási minták kutatásának. Magát a tervmintát nem definiálták. A fogalom maga évekig formalizálatlan maradt.[5]

Ez a könyv összesen 23 mintát mutat be, és a következő kategóriákba sorolja őket:

- létrehozási minta
- szerkezeti minta
- viselkedési minta

A szerzők kiemelik, hogy egyáltalán nem a valóságtól elrugaszkodott és elvont mintákat „találtak ki”, hanem csak összegyűjtötték a sokszor előforduló problémákat és a rájuk adott válaszokat, kielemezték őket, és a látottak alapján megpróbáltak általános megoldásokat nyújtani. A könyvben minden minta mellé leírták azt, hogy milyen felmerülő problémára adhat választ, alkalmazásának előnyeit és hátrányait, a megvalósítás lehetséges buktatóit, ismert felhasználásokat és példakódot, így teljes képet adva azok lehetséges használatának körülményeiről és eredményéről. Ez a könyv ma is referenciaként szolgál minden objektumorientált mintákkal kapcsolatos munkához.

Munkájuk alapján sok tervezési mintákkal foglalkozó közösség alakult, ezek közül talán a legismertebbek a Portland Patterns Repository[6] és a Hillside Group.[7] Mindkét oldal a tervezési minták népszerűsítését, a régi minták fejlesztését és új minták bemutatását tűzte ki céljául, a GoF könyvének szellemében. Az első kiadás óta eltelt több, mint tíz évben új könyvek sora is megjelent a témában, ezek közül a legismertebbek a Head First kiadó Design Patterns-e[8] és a Design Patterns Explained.[9]

A programtervezési minták definíciója, helye [szerkesztés]

A programtervezési minták a GoF definíciója szerint: „egymással együttműködő objektumok és osztályok leírásai, amelyek testre szabott formában valamilyen általános tervezési problémát oldanak meg egy bizonyos összefüggésben”. A szoftvertervezésben egy-egy problémára végtelen sok különböző megoldás adható, azonban ezek között kevés optimális van. Tapasztalt szoftvertervezők, akik már sok hasonló problémával találkoztak, könnyen előhúzhatnak egy olyan megoldást, amely már kipróbált és bizonyított. Kezdő fejlesztőknek viszont jól jön, ha mindazt a tudást és tapasztalatot, amit csak évek munkájával érhetnek el, precízen dokumentálva kézbe vehetik, tanulhatnak belőle és az általa bevezetett kifejezésekkel könnyebben beszélhetik meg egymás között az ötleteiket. A programtervezési minták ilyen összegyűjtött tapasztalatok, amelyek mindegyike egy-egy gyakran előforduló problémára ad általánosított választ. Azonban mindezek mellett még számos előnyük van:

- lerövidítik a tapasztalatszerzési időt. A programtervezési mintákat nem „feltalálták”, hanem a gyakran előforduló problémákra adott optimális válaszokat gyűjtötték össze, ezáltal olyan megoldásokat adtak, amelyekre előbb-utóbb a legtöbb fejlesztő magától is rájönne – csak esetleg jóval később. Természetesen nem kizárt, hogy léteznek jobb, hatékonyabb megoldások, és ritka amikor egy-egy mintát pontosan úgy lehet alkalmazni egy problémára, ahogy az a könyvekben le van írva, de mindenképpen érdemes megismerni őket, ha másért nem is, hogy elsajátíthassunk valamennyit a szerzők látásmódjából.
- lerövidítik a tervezési időt. Az összes minta jól dokumentált, könnyen újrafelhasználható, így ha egyszer alkalmazzuk őket, jó eséllyel egy hasonló problémánál újra eszünkbe fognak jutni az összes előnyükkel és hátrányukkal együtt. Így azonnal hatékony, rugalmas megoldást adhatunk és megkímélhetjük magunkat sok tervezéstől és esetleges újratervezéstől. Ráadásul a minták után található „következmények” rész elősegíti, hogy teljesebb képet kapjunk az alkalmazás hatásairól is.
- közös szótárat ad a fejlesztők kezébe. Ez megkönnyíti az egymás közti kommunikációt és a program dokumentálását is, hiszen könnyebb úgy beszélni egy probléma megoldásáról, ha van egy közös alap, ahonnan indulunk vagy amihez lehet hasonlítani az új terveket.
- magasabb szintű programozást tesz lehetővé. Mivel ezek a minták elterjedtségük miatt már kiállták nagyon sok programozó próbáját, feltehetőleg az optimális megoldást tartalmazzák a problémára.

A tábla-adat kapu elkülöníti az adatok elhozását és felhasználását egy kapu objektum segítségével. A kapu felhasználói függetlenek az adatok tárolásának módjától.[10]

A hajlékonyság elérése érdekében a tervminták további szinteket vezetnek be, amelyekkel fellazítják a kapcsolatokat. Ezzel javítják a módosíthatóságot, de a nem funkcionális tulajdonságok romlanak. Alapvető szabály, hogy ne adjunk olyan flexibilitást, ami valószínűleg nem lesz kihasználva. Nem megfelelő használatuk csak bonyolultabbá teszi a programot.[11]

A back-off programtervezési minta éppen az erőforrások használatának csökkentésére való.[12] A minta külön szálat indít, ami elvégzi a várakozó feladatokat, és rendszeresen ellenőrzi a feladatsort. Ha nincs várakozó feladat, akkor alszik. Ez a minta szűkös, illetve fizetős erőforrások esetén hasznos.

A programtervezési minták nem az egyetlen, a programozáshoz kapcsolható minták. Általában háromféle csoportosítást szoktak alkalmazni:[13]

- Menedzsmenti (Managerial) minták : folyamatok és emberek kezelésének szintje
- Architekturális (Architectural) : rétegek, erőforrások szintje
- Tervezési (Design) : kódolási gyakorlatok, osztályok, objektumok szintje

Ezen kívül szokták használni a programozási minták (programming pattern) kifejezést azokra a programtervezési mintáknál kisebb mintákra, amelyek egy objektum belsejének részleteivel vagy egy nyelv elemeinek használatával foglalkoznak. A sor-adat kapu egy tervminta, amiben egy objektum egy adatbázis sor számára szolgál kapuként.[14]

Mivel a tervmintákat minden újabb programba be kell programozni, ez visszalépést jelent a kód újrafelhasználása terén. Kutatók (köztük Meyer és Arnout) foglalkoznak azzal, hogy a tervmintákból komponenseket készítsenek.[15]

Szerkezete [szerkesztés]

A tervezési minták leírása több szakaszból áll. Ezek közül a legfontosabbak a szerkezet, a résztvevők és az együttműködés. Ezek írják le magát a mikroszerkezetet, amit alkalmazni lehet bizonyos problémák megoldására. A mikroszerkezet a program egyes egységeiből, mint osztályok és metódusok, és kapcsolataiból áll.

Vannak törekvések arra, hogy konkrétabb, bizonyos célokra szolgáló mintákat találjanak. Ide tartoznak a felhasználói interfészek,[16] az információmegjelenítés,[17] a biztonságos tervezés,[18] a biztonságos használat,[19] Web design [20] a webdesign és az üzleti modell terv.[21]

Egyes mintákat a programozók átalakítanak, hogy jobban illeszkedjenek az éppen megoldani kívánt problémához. Ezek a minták angolul amelioration pattern néven ismertek, és akár antimintának is tekinthetők, mert a továbbfejlesztő azt hihetné, hogy itt az eredeti mintát használták, pedig mégsem.

A Pattern Languages of Programming Conference évenkénti gyűlései és kiadványai sok tartományspecifikus mintát mutatnak be.[22]

Tervezési minták osztályozása [szerkesztés]

A tervezési minták a modulokat és kapcsolataikat szervezik. Alacsonyabb szintűek, mint az architekturális minták, amelyek a teljes rendszer általános felépítését jellemzik.[23]

Több különböző tervminta létezik, például:[24][25]

- Algoritmus stratégia minták
  Magas szintű stratégiák, amelyek leírják, hogyan kell algoritmust szervezni egy adott architektúrára.
- Számítástervezési minták
  A kulcsfontosságú számítások megkeresését célozzák.[26][27]
- Végrehajtási minták
  A végrehajtás alacsonyabb szintjén címzésekkel, feladatok végrehajtásának szervezésével, optimalizálásával, szinkronizálásával foglalkozó minták.
- Implementációs stratégia minták
  A forráskód implementációjában támogatják a program szervezését, és a párhuzamos programozás számára fontos adatszerkezetek felépítését.
- Szerkezeti tervezési minták
  Az alkalmazás globális struktúrájával foglalkoznak.

A tervmintákat Gammáék eredetileg három kategóriába csoportosították: létrehozási minták, szerkezeti minták, és viselkedési minták, amihez használták a delegálás, az aggregálás, és a konzultáció fogalmát. Objektumorientált környezetben az öröklődés, polimorfizmus és az absztrakt ősosztály fogalmait is felhasználják, habár az absztrakt osztályok többnyire interfészek is lehetnek. Egyes szerzők elkülönítik az architekturális tervezési mintákat is, mint a modell-nézet-vezérlő.

Létrehozási minták

Név	Leírás	*Design Patterns*	*Code Complete* (könyv)[28]	Egyéb
Absztrakt gyár (Abstract factory)	Létrehoz egy interfészt egymástól függő vagy valamilyen módon összekapcsolódó objektumok *családjainak* anélkül, hogy megadnánk a konkrét osztály típusát.	Igen	Igen	N/A
Építő	Egy objektum felépítési folyamattal több, különböző szerkezetű objektumok létrehozására. A létrehozás folyamata független az objektum ábrázolásától.	Igen	Nem	N/A
Gyártó metódus (Factory method)	Meghatároz egy interfészt egy objektum létrehozására, de a leszármazottakra van bízva az objektum típusának meghatározása. Segítségével elkerülhető, hogy alkalmazás specifikus osztályokat rakjunk a kódba. (Kapcsolódó: A kontroll megfordítása)[29]).	Igen	Igen	N/A
Lusta inicializáció	Objektum létrehozás, költséges számítás, stb. késleltetése addig, amíg az objektum/érték nem válik szükségessé.	Igen	Nem	PoEAA[30]
Többke	Hasonlóan az egyke tervezési mintához, egy példány készítését teszi lehetővé, viszont nem alkalmazásonként hanem kulcsonként. A multiton osztály tartalmaz egy mapot a példányokról (kulcs-érték párok).	Nem	Nem	N/A
Objektumkészlet	Objektumok újrafelhasználása. Objektumok példányosítása és törlése költséges művelet lehet, ezért előre legyártott objektumokat használ az alkalmazás, és miután egy szükségtelenné vált, visszahelyezi azt az objektumkészletbe. Tipikus példája a szálkészlet, amely processzek által használatos szálakat tárol.	Nem	Nem	N/A
Prototípus (Prototype)	Meghatároz egy előzetes mintát az objektumok létrehozásához, és később ez kerül másolásra. Gyakran használjuk, ha az objektum pontos típusa csak futásidőben derül ki.	Igen	Nem	N/A
Egyke (Singleton)	Biztosítja, hogy az osztályból csak egy példány készül, és biztosít egy publikus hozzáférést ehhez a példányhoz.	Igen	Igen	N/A

Szerkezeti minták

Név	Leírás	*Design Patterns*	*Code Complete* (könyv)[28]	Egyéb
Illesztő minta	Egy felület átalakítása, illesztése egy másikra. Összeférhetetlen osztályok együttműködését teszi lehetővé.	Igen	Igen	N/A
Híd	Lehetővé teszi az absztrakció (interfész) és az implementáció szétválasztását, így a kettő külön változtatható egymástól függetlenül.	Igen	Igen	N/A
Összetétel	Az objektumok faszerkezetbe való ábrázolásának megvalósítása. Egységesíti a kezelését az objektumnak és objektumok összetételének.	Igen	Igen	N/A
Díszítő	Lehetővé teszi az absztrakció változtatása nélkül további funkciók, felelősségi körök dinamikus hozzáadását.	Igen	Igen	N/A
Homlokzat	Egy egységes magasabb szintű interfészt hoz létre egy alrendszer számos interfészének. Leegyszerűsíti a bonyolult rendszert, általa könnyebben használhatók az alrendszer funkciói.	Igen	Igen	N/A
Pehelysúlyú minta	Sok egyforma objektum erőforrásaival hatékonyan gazdálkodik oly módon, hogy egy külön objektumban tárolja az újrahasznosítható (ismétlődő) funkciókat, tulajdonságokat.	Igen	Nem	N/A
Front vezérlő	Web alkalmazásokhoz kapcsolódó tervezési minta. A beérkező kérések feldolgozásának és kiszolgálásának vezérléséhez használt módszer.	Nem	Igen	N/A
Modul	Egy sor egymással összefüggő elemet (mint az osztályok, eljárások stb.) csoportosít egy közös konceptuális entitásba (modulba).	Nem	Nem	N/A
Helyettes	Egy másik objektum elfedésére, helyettesítésére alkalmazott tervezési minta.	Igen	Nem	N/A
Iker[31]	Lehetőséget ad a többszörös öröklődés megvalósítására olyan programozási nyelvekben, ahol ez nem támogatott.	Nem	Nem	N/A

Viselkedési minták

Név	Leírás	*Design Patterns*	*Code Complete* (könyv)[28]	Egyéb
Blackboard	Egy általánosított observer, amely engedélyez több írót és olvasót. Rendszerszintű az információ áramlása.	Nem	Nem	N/A
Felelősséglánc	Az üzenet vagy kérés küldőjének függetlenítése a fogadótól. Megvalósítása felelősségláncok kialakításával történik. Lényegében egy láncolt lista, amin a kérelem végig halad mindaddig, amíg egy objektum le nem tudja kezelni.	Igen	Nem	N/A
Parancs	Kérelmek objektumba ágyazása. Ezáltal a klienseknek különböző parancsokat adhatunk át, amit naplózhatunk, sorba rendezhetünk és a visszavonást kezelhetjük le. Azaz az egyes kérelmek teljesen kivizsgálhatóak és hatálytalaníthatók lesznek.	Igen	Nem	N/A
Értelmező	Meghatározza az adott nyelv értelmezőjének a forráskód értelmezésének nyelvtanát, szabályait.	Igen	Nem	N/A
Iterator	Lehetővé teszi egy lista aggregált objektum elemeinek szekvenciális elérését anélkül, hogy láthatóvá tenné a mögöttes reprezentációját.	Igen	Igen	N/A
Közvetítő	Több egymással kapcsolatot tartó objektum kommunikációjának egységbe (objektumba) zárása. Definiál egy interfészt, amin keresztül objektumokkal kommunikálhat.	Igen	Nem	N/A
Memento	Az egységbezárás megsértése nélkül a külvilág számára elérhetővé tenni az objektum belső állapotát. Így az objektum állapota később visszaállítható.	Igen	Nem	N/A
Üres objektum programtervezési minta	Null referenciák elkerülése érdekében null helyett egy alapértelmezett objektummal (null object) térnek vissza a metódusok.	Nem	Nem	N/A
Megfigyelő	Meghatároz egy egy-a-többhöz függőséget objektumok között. Egy adott objektum módosulásáról automatikus értesítő információt küld a tőle függő objektumoknak, amik ezek alapján frissülnek.	Igen	Igen	N/A
Szolga	Osztályok csoportjának biztosít funkciókat, melyeket elvégez az osztályok helyett. Az adott funkciók csak a szolga osztályban vannak megvalósítva.	Nem	Nem	N/A
Specifikáció	Egyesíti az alkalmazás üzleti logikáját Boole-algebra-szerűen.	Nem	Nem	N/A
Állapot	Az objektum viselkedése megváltoztatható a belső állapottól függően.	Igen	Nem	N/A
Stratégia	Egységbe zárt, azonos interfésszel rendelkező algoritmusok dinamikus cserélgetése. Lehetővé teszi, hogy az algoritmus az őt használó kliensektől függetlenül változzon.	Igen	Igen	N/A
Sablonfüggvény	Az algoritmus vázának megvalósítása oly módon, hogy bizonyos közbenső végrehajtási lépések specializálhatók legyenek az alosztályokban.	Igen	Igen	N/A
Látogató	Lehetőséget ad új funkciók hozzáadására anélkül, hogy megváltoztatnánk az osztályok szerkezetét.	Igen	Nem	N/A

Konkurenciaminták

Név	Leírás	*POSA2* (könyv)[32]	Egyéb
Aktív objektum	Függetleníti a metódushívást a futtatásától. Célja a konkurencia megvalósítása aszinkron metódushívással és egy ütemezővel a kérések feldolgozásához.	Igen	N/A
Balking (akadályozó)	Csak akkor futhat le egy függvény az objektumban, hogyha az objektum egy bizonyos állapotában van.	Nem	N/A
Kötési tulajdonságok (Binding properties)	Több megfigyelő (observer) egyesítése annak érdekében, hogy több objektum tulajdonságait szinkronizálni, koordinálni lehessen.	Nem	N/A
Duplán ellenőrzött zárolás	Megosztott erőforrások többszálú hozzáférését teszi lehetővé. Először teszteli, hogy rá lehet-e lockolni az adott lockra, és csak utána teszi meg.	Igen	N/A
Eseményalapú aszinkron minta	Több szálon futó alkalmazások szálkezelési problémáit célozza meg.[33]	Nem	N/A
Őrzött felfüggesztés	Olyan műveleteket kezel, amely futásához a lock megszerzése mellett bizonyos feltételeknek is teljesülniük kell.	Nem	N/A
Join	Olyan magas szintű programozási modell, ami által konkurens, paralel és elosztott programokat írhatunk, üzenetek átadásával.	Nem	N/A
Lock (zár)	Egy szál zárat (lock-ot) tehet a forrásra, megakadályozva ezzel azt, hogy más szálak hozzáférjenek vagy módosítsák azt.	Nem	PoEAA[30]
Üzenetküldő (MDP)	Lehetővé teszi az alkalmazások, komponensek közötti üzenetváltást.	Nem	N/A
Monitor objektum	Egy olyan objektum, amely segítségével zárolni lehet objektumokat, amíg inkonzisztens állapotban vannak. Monitorokat használva fellép a holtpont (deadlock) lehetősége.	Igen	N/A
Reactor	A reactor objektum aszinkron interfészt biztosít olyan forrásoknak, amelyeknek szinkronban kell futniuk.	Igen	N/A
Író/olvasó lock	Lehetővé teszi egy objektum egyidejű olvasását, viszont az íráshoz egyszerre csak egy szál férhet hozzá.	Nem	N/A
Ütemező	Ütemezi, irányítja, hogy mikor melyik szál futtathat le egy adott egy szálon futtatandó kódot.	Nem	N/A
Szálkészlet	Egy adott mennyiségű, sorban (queue) elhelyezett, előre legyártott szál, amelyekkel elkerülhető a szálak költséges legyártása és eltakarítása. Az Objektumkészlet minta egy speciális változata.	Nem	N/A
Szálspecifikus tároló	Statikus (globális) memória egy adott szálnak.	Igen	N/A

Dokumentáció [szerkesztés]

A tervminta dokumentációja leírja, hogy milyen körülmények között használták a mintát, milyen kényszerek formálták a körülményeket, és hogyan sikerült megoldani a helyzetet. A tervminták dokumentálására nincs egy szabványos forma; a különböző szerzők különféleképpen írták le őket. Azonban Martin Fowler úgy találta, hogy egyes formák népszerűbbek, mint mások; az új leírásokhoz ezeket ajánlja. Gammáék leírása ezek között van:

- Név, osztályozás: Azonosításra alkalmas, leíró név.
- Cél: Miért használjuk a mintát, mi célt szolgál.
- Más nevek: Alternatív elnevezések.
- Motiváció: Egy kontextusba ágyazott probléma, amit a tervmintával lehet megoldani.
- Alkalmazhatóság: Milyen körülmények között használható a minta.
- Szerkezet: A minta szerkezetének grafikus megjelenítése osztály- és interakciós diagramokkal.
- Résztvevők: A mintában szereplő osztályok és objektumok leírása, szerepük meghatározása.
- Együttműködés: Az interakciók leírása.
- Következmények: Az eredmények, mellékhatások, előnyök és hátrányok kifejtése.
- Implementáció: Az implementáció bemutatása.
- Példa kód: A tervminta megvalósításának példája egy programozási nyelven.
- Ismert használatok: A tervminta használata valós környezetben.
- Rokon minták: Más, hasonló, kapcsolódó minták, és a tárgyalt minta összehasonlítása velük.

Elemzés [szerkesztés]

A tervmintákat többen is elemezték és kritizálták.

A tervmintákra többnyire azért van szükség, mert az adott programozási nyelven nem célszerű az adott problémát másként megoldani, vagy mert nincs egyszerűbb, átláthatóbb megoldás rá. Ez tekinthető az adott programozási nyelv hibájának is. Peter Norvig megmutatta, hogy Gammáék Design Patterns könyvében leírt 23 minta közül 16 egyszerűsíthető Lisp és Dylan nyelveken.[34] Hasonló megfigyeléseket tett Hannemann és Kiczales, akik AspectJ-ben, egy aspektusorientált nyelvben valósítottak meg tervmintákat. A kódszintű függőségeket a 23 közül 17-ből ki tudták küszöbölni, és megmutatták, hogy aspektusorientált programozással a minták egyszerűsíthetők.[35] Paul Graham az eredményeket egy esszében (Revenge of the Nerds) foglalta össze.[36]

Jegyzetek [szerkesztés]

1. ↑ Szabolcsi Judit: Szoftvertechnológia - Programtervezési minták (Design Pattern)[halott link]
2. ↑ Smith, Reid (1987. október 1.). „Panel on design methodology”. *OOPSLA '87 Addendum to the Proceedings*. doi:10.1145/62138.62151.
3. ↑ Beck, Kent (1987. szeptember 1.). „Using Pattern Languages for Object-Oriented Program”. *OOPSLA '87 workshop on Specification and Design for Object-Oriented Programming*. **Hozzáférés: 2006. május 26.**
4. ↑ Desing Patterns (könyv), i. m.
5. ↑ Baroni, Aline Lúcia (2003. június 1.). „Design Patterns Formalization”, Nantes, Kiadó: École Nationale Supérieure des Techniques Industrielles et des Mines de Nantes.
6. ↑ *The Portland Patterns Repository*. (Hozzáférés: 2009. november 23.)
7. ↑ *The Hillside Group*. (Hozzáférés: 2009. november 23.)
8. ↑ Head First Design Patterns, i. m.
9. ↑ Desing Patterns Explained, i. m.
10. ↑ Fowler, Martin: *P of EAA: Table Data Gateway*. (Hozzáférés: 2009. február 11.)
11. ↑ McConnell, Steve. *Code Complete: A Practical Handbook of Software Construction, 2nd Edition*, 105. o. (2004)
12. ↑ *AzureProviders*. [2017. augusztus 21-i dátummal az eredetiből archiválva]. (Hozzáférés: 2017. augusztus 21.)
13. ↑ AntiPatterns, i. m.
14. ↑ Fowler, Martin: *P of EAA: Row Data Gateway*. (Hozzáférés: 2009. február 11.)
15. ↑ Meyer, Bertrand (2006. július 1.). „Componentization: The Visitor Example”. *IEEE Computer* **39** (7), 23–30. o, Kiadó: IEEE. DOI:10.1109/MC.2006.227.
16. ↑ Laakso, Sari A.: *Collection of User Interface Design Patterns*. University of Helsinki, Dept. of Computer Science, 2003. szeptember 16. (Hozzáférés: 2008. január 31.)
17. ↑ Heer, J. (2006). „Software Design Patterns for Information Visualization”. *IEEE Transactions on Visualization and Computer Graphics* **12** (5), 853–60. o. (2008). DOI:10.1109/TVCG.2006.178. PMID 17080809.
18. ↑ *Secure Design Patterns*. Software Engineering Institute (2009. január 2.)
19. ↑ *Garfinkel, Simson L. (2005), Design Principles and Patterns for Computer Systems That Are Simultaneously Secure and Usable*, <http://www.simson.net/thesis/>
20. ↑ *Yahoo! Design Pattern Library*. [2008. február 29-i dátummal az eredetiből archiválva]. (Hozzáférés: 2008. január 31.)
21. ↑ *How to design your Business Model as a Lean Startup?*. (Hozzáférés: 2010. január 6.)
22. ↑ Pattern Languages of Programming, Conference proceedings (annual, 1994—) [1]
23. ↑ Martin, Robert C.: *Design Principles and Design Patterns*, 2000 [2015. szeptember 6-i dátummal az eredetiből archiválva]. (Hozzáférés: 2017. május 26.)
24. ↑ https://sourcemaking.com/design_patterns
25. ↑ *Design Patterns | Object Oriented Design*. *www.oodesign.com*. (Hozzáférés: 2017. április 8.)
26. ↑ *Category:Computational Thinking Patterns – Scalable Game Design wiki*. *sgd.cs.colorado.edu*. [2016. január 5-i dátummal az eredetiből archiválva]. (Hozzáférés: 2015. december 26.)
27. ↑ *Introduction to Software Engineering/Architecture/Design Patterns – Wikibooks, open books for an open world*. *en.wikibooks.org*. (Hozzáférés: 2015. december 26.)
28. ↑ a b c McConnell, Steve. Design in Construction, *Code Complete*, 2nd, Microsoft Press, 104. o. (2004. június 1.). ISBN 978-0-7356-1967-8 „Table 5.1 Popular Design Patterns”
29. ↑ *Design Patterns: Dependency injection*. (Hozzáférés: 2011. április 13.) „The use of a factory class is one common way to implement DI.”
30. ↑ a b Fowler, Martin. *Patterns of Enterprise Application Architecture*. Addison-Wesley (2002). ISBN 978-0-321-12742-6
31. ↑ *Twin – A Design Pattern for Modeling Multiple Inheritance*
32. ↑ Schmidt, Douglas C.. *Pattern-Oriented Software Architecture, Volume 2: Patterns for Concurrent and Networked Objects*. John Wiley & Sons (2000). ISBN 0-471-60695-2
33. ↑ Christian Nagel, Bill Evjen, Jay Glynn, Karli Watson, and Morgan Skinner. Event-based Asynchronous Pattern, *Professional C# 2008*. Wiley, 570–571. o. (2008). ISBN 0-470-19137-6
34. ↑ Norvig, Peter (1998). „Design Patterns in Dynamic Languages”..
35. ↑ Hannemann, Jan (2002). „Design pattern implementation in Java and AspectJ”..
36. ↑ Graham, Paul (2002). „Revenge of the Nerds”.. Hozzáférés: 2012. augusztus 11.

Források [szerkesztés]

- Alan Shalloway, James R. Trott. *Desing Patterns Explained*. Addison-Wesley, 337. o. (2002). ISBN 0-201-71594-5
- Alexander, Christopher. *A Pattern Language*. Oxford University Press (1977). ISBN 0195019199
- Gamma, Helm, Johnson & Vlissides. *Design Patterns (könyv)*. Addison-Wesley (1994). ISBN 0-201-63361-2
- Eric Freeman, Elisabeth Freeman, Kathy Sierra, Bert Bates. *Head First Design Patterns*. O'Reilly Media, 638. o. (2004). ISBN 0-596-00712-4
- William J. Brown, Raphael C. Malveau, Hays W. "Skip" McCormick , Thomas J. Mowbray. *AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis*. Wiley, 336. o. (1998). ISBN 0-471-19713-0
- „The use of anti-patterns in human computer interaction: wise or Ill-advised?”. *Proceedings of the 2004 annual research conference of the South African institute of computer scientists and information technologists on IT research in developing countries (SAICSIT '04)*.

Fordítás [szerkesztés]

Ez a szócikk részben vagy egészben a *Software design pattern* című angol Wikipédia-szócikk fordításán alapul. Az eredeti cikk szerkesztőit annak laptörténete sorolja fel. Ez a jelzés csupán a megfogalmazás eredetét és a szerzői jogokat jelzi, nem szolgál a cikkben szereplő információk forrásmegjelöléseként.

Ez a szócikk részben vagy egészben az *Amelioration pattern* című angol Wikipédia-szócikk fordításán alapul. Az eredeti cikk szerkesztőit annak laptörténete sorolja fel. Ez a jelzés csupán a megfogalmazás eredetét és a szerzői jogokat jelzi, nem szolgál a cikkben szereplő információk forrásmegjelöléseként.

A lap utolsó módosítása: 2024. január 15., 05:04

A lap szövege Creative Commons Nevezd meg! – Így add tovább! 4.0 licenc alatt van; egyes esetekben más módon is felhasználható. Részletekért lásd a felhasználási feltételeket.